The True Lethality Engine

Predicting the real danger of D&D 5e combat from 34,907 recorded encounters

Daniele · Francesca · Stefano · Antonietta

Statistical Machine Learning, Final Project. Sapienza Università di Roma, A.Y. 2025/2026

`https://dnd-appraising.streamlit.app` `https://github.com/danpinocontrollino/project-dnd`

Abstract

Dungeons & Dragons fifth edition (D&D 5e) assigns every monster a *Challenge Rating* (CR), a difficulty score chosen by the designers and, to our knowledge, never validated against actual play. In this project we validate it, and then replace it. Starting from 153,829 turn snapshots of real online games (the FIREBALL corpus) we build 34,907 encounter-level observations from 1,462 campaigns, parse offensive statistics directly out of the monster statblock text, and train a calibrated, monotonicity-constrained XGBoost (eXtreme Gradient Boosting) model of $\eta(x) = P(\text{party wins} \mid x)$. Inverting η by binary search gives the *True Lethality Level*: the party level at which a balanced party of four reaches a target win rate. All validation is grouped by campaign, which is the real unit of information leakage in this data; the held-out area under the receiver operating characteristic curve (ROC–AUC) is 0.657 (bootstrap 95% confidence interval, CI, $[0.638, 0.674]$) with a Brier score of 0.139 $[0.134, 0.145]$ against an 83% base rate. Observational play turns out to be contaminated by what we call Dungeon Master (DM) mercy: fights that the damage arithmetic declares hopeless were still recorded as wins 84.5% of the time. The engine therefore adds three explicit domain guards, and the constants of the survival guard are estimated from 490,640 simulated fight-to-the-death battles rather than chosen by hand. A ridge logistic model actually beats the production model on observational AUC (0.655 vs. 0.614) and is rejected anyway, because its confounded coefficients make it unusable for the interventional questions the application asks. The final product is a live encounter appraiser that measures the rulebook against the data, with the rulebook's own procedure implemented faithfully on our side.

1 Introduction

In D&D 5e a party of 3 to 6 heroes (levels 1 to 20) fights monsters controlled by a Dungeon Master (DM). The official difficulty currency is the Challenge Rating: a CR-k monster is supposed to challenge four level-k heroes, and the *Dungeon Master's Guide* (DMG, p. 82) grades a whole encounter by comparing multiplier-adjusted monster experience points (XP) against per-level thresholds [2]. CR is a design intention. It compresses offense, defense, special traits and action economy into one printed number, and its failure modes are folklore among players: high-CR solo monsters that fold to action economy, cheap swarms that kill, trait synergies (a flying monster against a melee party) that no single score can express. Nobody had measured any of this against play, so we asked a precisely defined question:

What is the calibrated probability $\eta(x)$ that a given party wins a given encounter x, and at which party level L does a balanced party of four reach a chosen target win rate (default 65%)?

We find L by binary search on η over levels $[1, 20]$ (18 iterations, result rounded to quarter levels, implemented in `lethality_engine.lethality_appraisal`) and call it the **True Lethality Level**. Boundary cases receive explicit verdicts instead of an invented number: *trivial* when η is above the target already at level 1, and *beyond deadly* when it is below the target even at level 20, in which case we also report the level-20 ceiling.

Contributions. (i) An encounter-level dataset distilled from real play, with offensive statistics parsed from statblock text. (ii) A calibrated, shape-constrained model, validated with campaign-grouped splits and bootstrap uncertainty. (iii) A diagnosis of estimand mismatch in observational game data (DM mercy) and a three-guard architecture; one guard is calibrated by external simulation. (iv) A benchmark of the course toolbox (penalized logistic regression, random Fourier features (RFF) kernel machine, maximum mean discrepancy (MMD) two-sample test) that yields a concrete model-selection lesson. (v) A deployed application that compares the model with the DMG's own procedure, implemented honestly.

2 Data

2.1 From chat logs to encounters

FIREBALL [1] collects real games played on Discord through the Avrae bot. Our ETL script (`parse_fireball.py`) reads 1,471 JSONL logs, i.e. 153,829 turn snapshots of which 134,475 are usable. Outcomes are inferred from terminal hit-point (HP) strings ("Dead", "0/45"); 85,778 unresolved fights and 9,215 encounters with no bestiary match are dropped. The result is **34,907 resolved encoun-**

ters from **1,462 campaigns**, with a party win base rate of **83.0%**, a number that shapes every downstream decision. Each row carries 29 raw columns: party level, size and four role flags (healer, tank, arcane, martial DPS, mapped from character classes); roster aggregates computed as count-weighted means for continuous statistics, maxima for binary traits and for apex threats, and sums for pooled HP, damage per round (DPR) and XP; ten monster trait flags; and the outcome. Party levels are clipped to $[1, 20]$ because multiclass strings contain spurious digits, and rosters to 30 because mass-summon logs reach 241 tokens on the field.

2.2 Offensive statistics from statblock text

Monster manuals list HP and armor class (AC) but no machine-readable offense, which had forced an earlier version of the model to use CR itself as a proxy for damage. The module `monster_offense.py` parses offense from the System Reference Document (SRD) statblock text: attack bonuses (`+9 to hit`), printed average damage (`12 (2d6+5)`), multiattack routines ("makes *three* attacks"), save DCs, and spell lists priced by a 30-spell Player's Handbook (PHB) damage table. Parsing is per action, not on the pooled text: weapon attacks build *sustained* DPR (multiattack count times mean damage per action, riders summed within an action), while save-based actions such as breath weapons are excluded from DPR and become the *burst* statistic, the single scariest action. The Lich, for example, parses to DPR 45 and burst 100 (*Power Word Kill*). Coverage is 321 of 762 monsters; the rest are imputed from the DMG design table by CR, and parsed DPR is clamped to $[0.25\times, 3\times]$ of that table's band so a regex accident cannot create a CR-1 monster with DPR 400. Unit tests pin two real bugs we fixed: versatile weapons were double-counted ("7 … or 8 two-handed" summed to 15; 24 monsters affected) and dragon breath leaked into the multiattack mean (Adult Red Dragon 72.6 → 58; hand-computed truth is roughly 45 to 48). Spot checks: Goblin 5 (true 5), Tarrasque 150 (true ≈148).

2.3 Implementing the rulebook fairly

Since we claim to beat the DMG, we must implement it correctly. `official_encounter_estimate()` in `lethality_engine.py` runs the full p. 82 procedure: total XP, the count-multiplier ladder ($\times 1$, $\times 1.5$, $\times 2$, $\times 2.5$, $\times 3$, $\times 4$) with the p. 83 party-size step adjustment, and an inverse XP lookup (`xp_to_cr`) to the equivalent single-monster CR. Six CR-1 monsters are therefore reported as $1{,}200 \times 2 = 2{,}400$ adjusted XP, about one CR-6 monster, and not as "CR 1". Nine regression tests cover this code, including the DMG's own worked example, so any discrepancy we report measures the model against the rulebook's best effort.

3 Methods

3.1 Features

There are 45 features in five families: raw party and roster aggregates (15); offensive potency (5); power curves and ratios (6); trait-by-composition interactions such as *magic resistance* × *arcane party* (6); and combat mathematics plus the DMG budget (13). The combat-math family encodes the actual d20 rules: for instance the party hit chance is $\mathrm{clip}\big((21 + \mathrm{atk}_{\mathrm{party}} - \mathrm{AC}_{\mathrm{monster}})/20,\ 0.05,\ 0.95\big)$ with a level-dependent estimate of the party attack bonus, and symmetric for the monsters; from these we derive rounds-to-kill in both directions, their log ratio, and `burst_vs_pc_hp` (can one action delete a hero from full health?). All feature engineering lives in a single `sklearn` transformer (`DnDFeatureEngineer`) that is serialized inside the model pickle, so training and serving cannot compute different features.

3.2 Model and calibration

The production model is XGBoost [3] with 300 trees of depth 4, learning rate 0.149, subsample 0.68, column subsample 0.85, `min_child_weight` 7 and regularization $(\lambda, \alpha, \gamma) = (0.105, 0.091, 0.009)$, chosen by Optuna [4] with the grouped cross-validation (CV) AUC as objective (the study is persisted to SQLite and resumes across runs). We impose **45 monotone constraints**: positive for party level, party size and each role flag; negative for monster count, CR, HP, DPR, burst, attack bonus, save difficulty class (DC) and every hostile trait. The constraints double as the out-of-distribution (OOD) safety net, together with input clipping to legal 5e ranges inside the transformer. Probabilities are calibrated with Platt scaling [5] on *campaign-grouped* folds; this works because the pipeline preserves row order, so precomputed positional splits stay valid on the transformed matrix the calibrator sees. We first tried isotonic calibration and rejected it for a concrete reason: its piecewise-constant map collapsed the binary search onto plateau edges (one Lich and two Liches received the same level) while also measuring marginally worse (Brier 0.1394 vs. 0.1397).

3.3 Validation protocol

Encounters within one campaign share the party, the DM and the house rules, so the campaign is the unit of leakage. Every split is grouped by it: 4-fold `StratifiedGroupKFold` for model selection, a grouped 80/20 holdout (`GroupShuffleSplit`, seed 42; n_{train}=27,502, n_{test}=7,106), and the calibration folds themselves. Headline metrics carry 95% percentile-bootstrap confidence intervals (2,000 resamples of the test set; resamples with a single class are skipped). As a shift diagnostic we run a kernel two-sample test (unbiased MMD^2 [7], radial basis function (RBF) kernel with median-heuristic bandwidth, permutation null

with 200 permutations, 1,500 points per side) between train-campaign and held-out-campaign feature distributions: $\mathrm{MMD}^2 = 3.1 \times 10^{-4}$, $p = 0.12$. There is no detectable covariate shift, so the gap between grouped CV and holdout reflects campaign-level outcome variance, not distribution drift.

4 Guards and simulation

4.1 DM mercy

An early model rated *nineteen* CR-21 Liches as beatable by a level-8 party with 66% probability. The cause was in the data, not the code: in fights where the deterministic damage race says the party is deleted within one round, the logs still record a party win **84.5%** of the time (659 rows). DMs fudge rolls, retreats get logged as wins, reinforcements arrive. The logs answer "what happens at real tables"; the application asks "what happens if you fight to the death". This is an estimand mismatch, and no additional data collected the same way can fix it.

4.2 Three guards

First, the shape constraints and clipping described above. Second, *roster dominance*: count-weighted averages dilute, so a Lich with two ogres and six goblins has mean CR 2.94 against the Lich's own 21, and the unguarded model rated that larger fight *easier* (level 3.25 vs. 5.0). The engine therefore also scores every homogeneous sub-roster and returns the elementwise minimum of the win probabilities, enforcing the axiom that adding monsters can never help the party (`predict_win_for_parties`). Third, a *deathmatch cap*,

$$P(\text{win}) \;\le\; \min\bigl(\sigma(A\,s - C\ln k + B),\ \Lambda\bigr),$$

where s is the deterministic number of rounds the party survives and k the number of rounds it needs to kill the roster, both computed from the same feature mathematics the model trains on (k deliberately unclipped, so a 10,000-HP homebrew wall stays hopeless), and Λ is a lattice term described below. The survive term catches fast wipes; the $\ln k$ term catches slow attrition losses, a failure mode our first two guards missed entirely. Sign constraints on A and C keep the cap increasing in party level and decreasing in monster count, so the binary search and the dominance property both survive it.

4.3 Calibrating the cap by simulation

Hand-tuned constants would be a weakness in an exam and in production alike, so we estimated them externally. We vendored the public Monte Carlo engine of Battlecast (a 5e combat simulator that resolves initiative, movement, spell slots and conditions, with scripted play and no mercy; provenance and credits in `battlecast_bridge/PROVENANCE.md`) and ran three designed grids, 304 cells and **490,640 simulated battles** in total, with up to 2,000 trials per cell (the driver reduces trials adaptively for the largest rosters). The party is fixed to Fighter/Cleric/Wizard/Rogue. The grids: a 180-cell *guard* grid (boss CR $\{2, 5, 10, 15, 21\}\times$ count $\{1, 2, 4, 8, 12, 19\}\times$ level $\{1, 5, 9, 13, 17, 20\}$); a 100-cell *mercy* grid (25 SRD monsters spanning CR 0 to 10, four levels each); and a 24-cell *OOD* grid of HP- and AC-scaled clones. A binomially weighted logistic fit of simulated $P(\text{win})$ on both race features, over the guard and OOD grids together, gives

$$\sigma(0.19\,s - 2.09\ln k + 4.12).$$

Our first fit used the survive term alone, $\sigma(1.63\,s - 3.98)$, and it carried a blind spot the simulator exposed: one Lich against a level-5 party survives 4+ estimated rounds, so the survival-only cap sat at 0.95 while Battlecast had the party at **0.000** over 2,000 deathmatches. Surviving is not winning when 135 legendary hit points sit behind AC 17. Two time-to-kill (TTK) features cannot fully separate such cells (spell rotations and AoE are not priced by the damage math), so wherever simulated truth exists we serve it directly: Λ trilinearly interpolates the guard grid itself over (CR, count, level), made monotone at build time and abstaining below CR 2 (`battlecast_bridge/guard_lattice.json`). On grid cells the served probability *is* the simulated one. With all guards in place the Lich ladder reads $1 \to 11$, $2 \to 16.5$, and $3+ \to$ *beyond deadly* (four Liches: 7% at level 20; eight or more: 0.0%), against simulated 65%-crossings of $\approx$11 and $\approx$16.5 (Fig. 1).

Pairing both lenses on the mercy grid quantifies the bias (Fig. 2): correlation 0.842, with a crossover. The model sits about +0.28 above the simulator on hard fights (mercy inflation; only a handful of cells fall in this region, so the number is directional) and about -0.14 below on easy ones, which is the 83% ceiling produced by retreats and noisy labels. On the OOD grid the extremes agree exactly (HP$\times$20 clones win 0.000 to 0.003 of deathmatches) while fine mid-range ordering is only moderate (Spearman $\rho = 0.54$). Two caveats we state openly: the simulator implements 2024-SRD rules while our logs are 2014-era play, and its scripted optimal play makes deathmatch rates an upper bound.

5 Results

5.1 Held-out performance

Table 1 reports performance on held-out campaigns. Calibration is tight where the data mass lives (predicted 0.8 to 0.9) with mild overconfidence in the lowest bins (Fig. 3). TreeSHAP values, computed through XGBoost's native `pred_contribs` [9], rank party level, monster size, nova pressure (`burst_vs_pc_hp`), action economy (`num_monsters_total`) and party hit chance as the leading signals. In other words, the offensive statistics that the official bestiary does not even

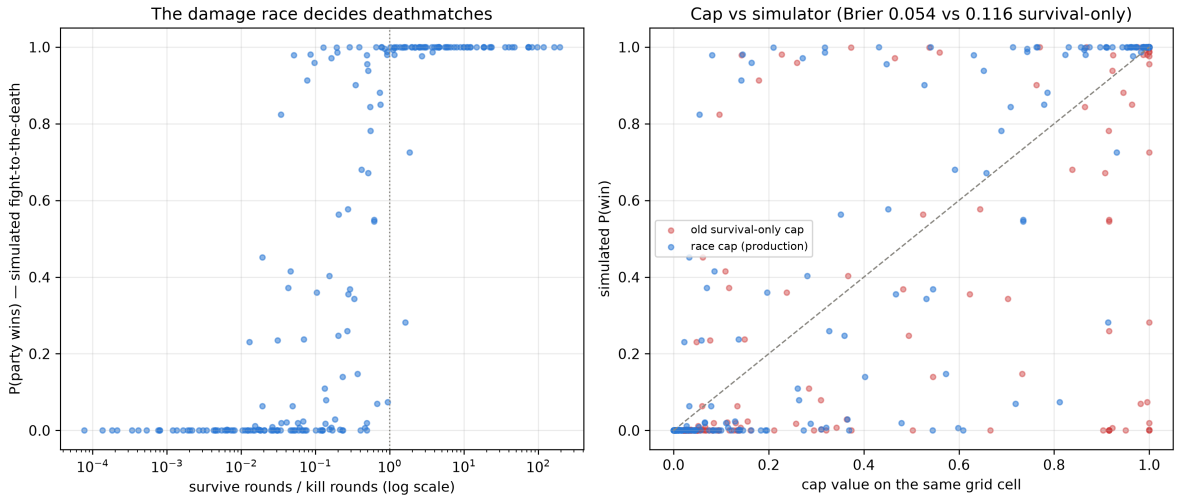

Figure 1: Deathmatch cells vs. the damage race (left) and cap values against simulated outcomes (right): the race fit against the survival-only guard it replaced.

Metric (held-out, n=7,106)	Value	95% CI
ROC–AUC	0.657	[0.638, 0.674]
Brier score	0.139	[0.134, 0.145]
Precision–recall AUC (PR–AUC)	0.874	—
Log loss	0.447	—
Grouped 4-fold CV AUC	0.601 ± 0.040	
Base rate P(win)	0.830	

Table 1: Production model, campaign-grouped evaluation.

publish are among the strongest predictors of real outcomes.

5.2 Prediction is not decision

Under identical grouped CV (Table 2), an ℓ_2-penalized logistic regression (C=0.03; the sweep over $C \in [3 \cdot 10^{-3}, 3]$ was nearly flat) *beats* the production model on observational risk. We promoted it to production as an experiment, and it failed immediately: it rated 8 Liches as trivial and a 10,000-HP monster as beatable. Its coefficients explain why. With no shape constraints the linear fit absorbs selection confounding: `num_monsters`, `total_dpr` and `burst` come out *positive* and `avg_party_level` comes out *negative*, meaning "stronger parties lose more", because stronger parties are served harder content (`figures/logistic_coefficients.png`). The application's queries are interventional ("same monster, more of them"), so we deploy the constrained model and accept roughly 0.04 AUC as the price of sane counterfactual behavior. A 13-axiom behavioral suite (`behavior_suite.py`: count monotonicity, roster dominance, level monotonicity, boundary verdicts, the full OOD flow through the CR predictor) now gates every model change, alongside 64 unit tests.

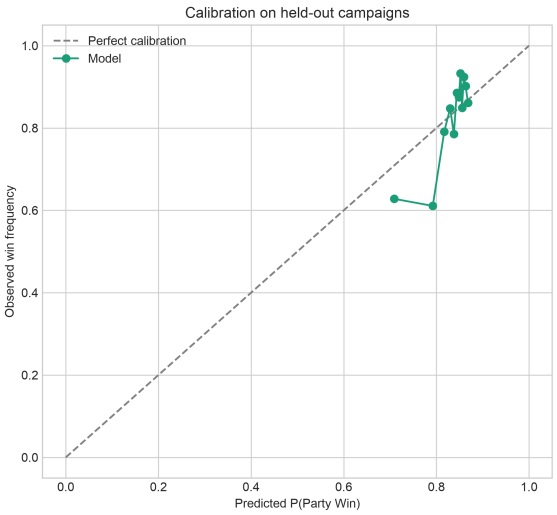

Figure 3: Reliability on held-out campaigns.

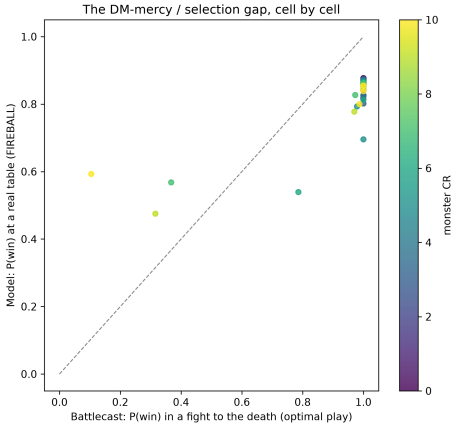

Figure 2: DM mercy quantified: model (table reality) vs. simulator (deathmatch), one point per encounter, color = CR.

5.3 Synthetic balancing ablation

Conditional Tabular Generative Adversarial Network (CTGAN) [8] synthetic losses (23,091 rows, fitted only on training campaigns to avoid leakage) used to balance the 83/17 classes degrade the same holdout to AUC 0.638 and Brier 0.186, and drag the mean predicted probability from 0.83 down to 0.61. With a proper loss and calibration, class imbalance was never the problem; resampling destroys calibration to fix a non-issue.

6 The application

A Streamlit appraiser (auto-deployed from `main`) offers three modes: the official bestiary (762 monsters, each offense value labeled with its provenance, parsed statblock or DMG table), a homebrew appraiser (a CR predictor answering "what the book would print", MAE 0.9, next to the True Lethality Level), and a mixed-roster encounter builder whose aggregation matches training exactly. Every appraisal shows the by-the-book verdict next to the model's, win curves for five party compositions, a party-size by level heatmap, the fairest matchups with official DMG tiers, and threat flags. The target win rate is a slider from 0.50 to 0.90: "fair" is a policy, not a constant. Reproducibility: `make retrain` runs data → training → 64 tests → 13 behavioral checks; `make battlecast` regenerates the simulation study; every training run is appended to `figures/experiments.jsonl` with its git commit.

7 Limitations

(i) *Observational ceiling*: dice variance, table tactics and unrecorded magic items bound what any feature set can see; at an 83% base rate, AUC 0.657 with confidence intervals is an honest number, not a weak one. (ii) *Estimand mismatch* is handled by an explicit, simulation-calibrated guard: a documented modeling choice with its own tests, not something learned end to end. (iii) *Simulator caveats*: 2024-SRD rules, optimized default builds, scripted play. (iv) *Thin support on hard fights*: 96 of 100 mercy-grid cells are deathmatch-easy. Future work: instrumenting a virtual tabletop (Foundry) for clean combat events, a 30-second post-encounter form with explicit retreat/fudge labels that would turn mercy into a censoring variable, and a replay grid of real logged encounters restricted to SRD rosters.

Model (identical grouped CV)	AUC ± sd	Brier
Ridge logistic	0.655 ± 0.031	**0.136**
RFF kernel logistic [6]	0.624 ± 0.013	0.142
XGBoost + constraints (ours)	0.614 ± 0.033	0.141

Table 2: Model comparison. The linear model wins the metric and loses the job.

8 Conclusions

Five lessons travel beyond the game. Validate by the unit of leakage, or every number is fiction. Calibrate when the product consumes probabilities, and measure the calibration instead of assuming it. A model with better AUC can be unusable; constraints encode the difference between prediction and decision. Know your estimand, because observational data cannot answer counterfactual questions, while simulation can, and can even estimate your priors. And the rulebook deserved a fair trial: honestly implemented, it still loses to 34,907 real fights.

References

[1] A. Zhu et al. FIREBALL: A Dataset of Dungeons and Dragons Actual-Play with Structured Game State Information. *ACL*, 2023.

[2] Wizards of the Coast. *Dungeon Master's Guide*, pp. 82–83, 274. 2014.

[3] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. *KDD*, 2016.

[4] T. Akiba et al. Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD*, 2019.

[5] J. Platt. Probabilistic Outputs for Support Vector Machines. *Advances in Large Margin Classifiers*, 1999.

[6] A. Rahimi and B. Recht. Random Features for Large-Scale Kernel Machines. *NeurIPS*, 2007.

[7] A. Gretton et al. A Kernel Two-Sample Test. *JMLR*, 13, 2012.

[8] L. Xu et al. Modeling Tabular Data using Conditional GAN. *NeurIPS*, 2019.

[9] S. Lundberg and S.-I. Lee. A Unified Approach to Interpreting Model Predictions. *NeurIPS*, 2017.